

Houzair Koussa

Software engineer @ Stratalis

Maintainer @ thephilosophicalcode.com

The Importance Of Fundamentals

About

Fundamentals - building blocks of software

Showcase the importance of *understanding & applying* these fundamentals

Focus on personal, practical & “real life” case studies

Agenda

1. Clarifying fundamentals
2. Justifying fundamentals
3. Fundamentals in practice
 - a. PostgreSQL dying at 16:45
 - b. Proxy detection
 - c. AI coding agents

Disclaimer

This is an opinion piece based on my personal experiences

The opinions I express might differ among engineers (they probably do)

So, ask around and stay curious

Clarifying fundamentals

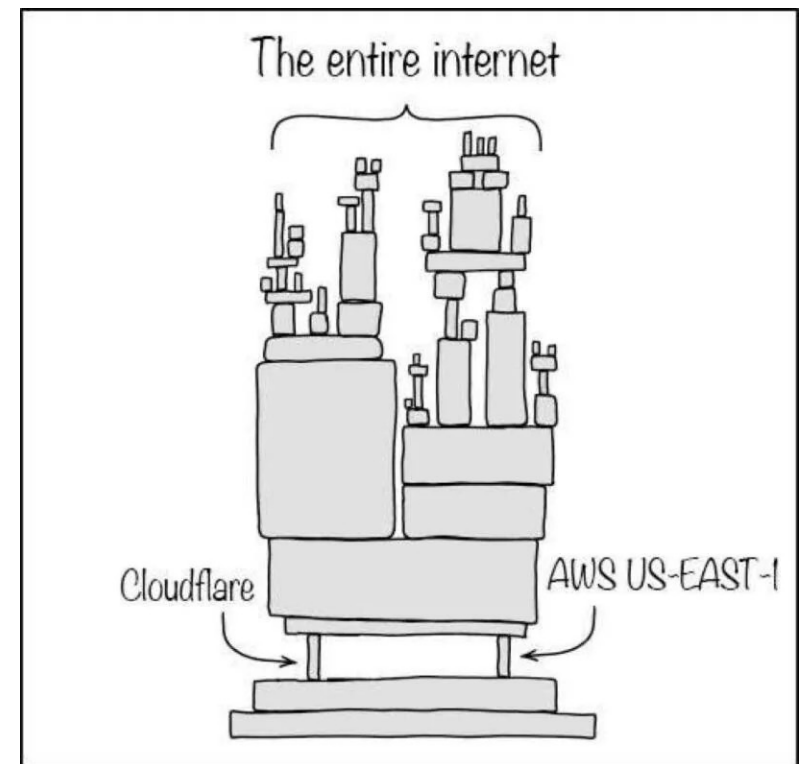
Layers of abstraction

Software is built upon layers of abstraction

Each layer depends on the one below

but each has its own building blocks

“Fundamentals” refer to each layer’s building blocks



Clarifying fundamentals

Fundamentals are relative

Crucially, each layer's fundamentals differ from other layers (even from those below)

“Python is just C” *but* Python has its own building blocks

Python cares about the GIL or async/await. C cares about pointers or memory

This applies across engineering roles as well - frontend, backend, infra, ...

Justifying fundamentals

Core justification

Irrespective of layer of abstraction, being able to *understand & apply* fundamentals makes you a better engineer

Justifying fundamentals

How we're gonna proceed

Overview of my current work across different layers of abstraction & domains

Take some recent events at work as case studies

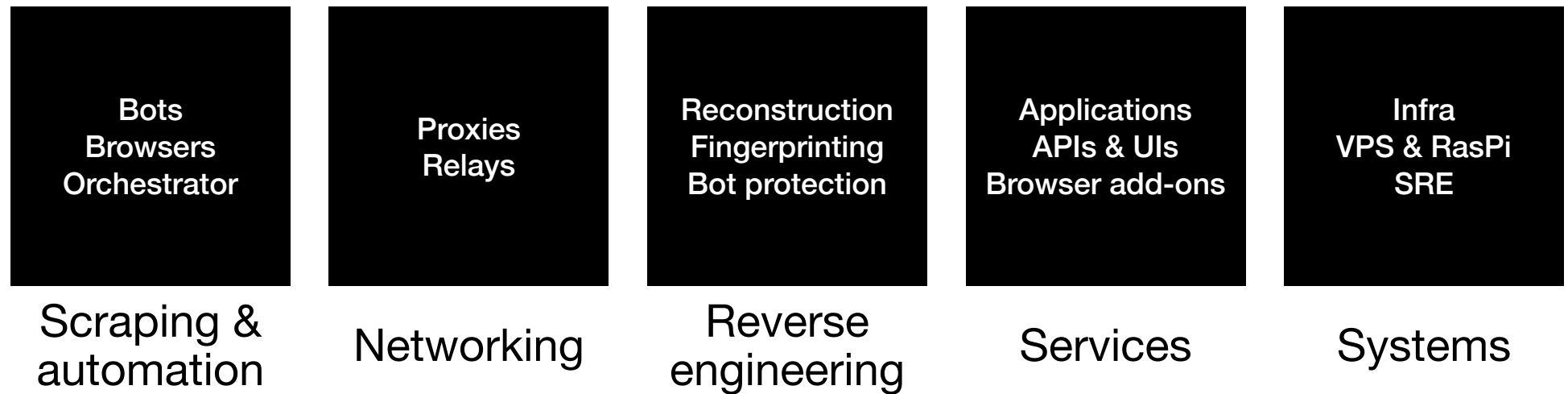
For each case study, I'll

- a. Briefly explain the fundamentals involved
- b. Showcase how they were applied

Fundamentals in practice

My current work - layers of abstractions across domains

Primary objective - engineer software for & around scraping data



Fundamentals in practice

My current work - engineering activities

Software development, architecture & operations

Research & discovery

Migration & adaptation

...

Fundamentals in practice

Case study 1 - overview

Layer of abstraction - Applications, infra & SRE

Domain - Services & systems

Activity - software development, architecture & operations

Fundamentals - processes, memory & disk

Fundamentals in practice

Processes, memory & disk

Roughly, a computer stores working data in memory to do stuff (aka RAM)

A process is a program in execution (like Chrome). Each process is allocated a piece of memory

Disk is where data is permanently stored (like your imminent assignment sitting on your SSD)

Fundamentals in practice

PostgreSQL dying at 16:45 - the story

At 16:45, multiple production services on a particular server returned 5xx's

For that server, our monitoring showed that

- a. A PostgreSQL production instance was down
- b. The RAM/memory usage was maxed out

Processes use memory. I nuked the process (a service app) maxing RAM out

Our OOM killer nuked PostgreSQL (which itself is just a process) as collateral

Fundamentals in practice

PostgreSQL dying at 16:45 - the fix

`some_list` is stored in memory

So, carelessly appending into it eats up memory

```
some_list = []
for i in SOME_HUGE_DATASET:
    # ...
    some_list.append(i)
```

Simple fix - let's write to disk instead

```
with open("some/path") as some_file_on_disk:
    for i in SOME_HUGE_DATASET:
        # doing stuff
        some_file_on_disk.write(i)
```

Adopted another OOM Killer

Added memory limits to the app container

Fundamentals in practice

Case study 2 - overview

Layer of abstraction - Proxies & relays

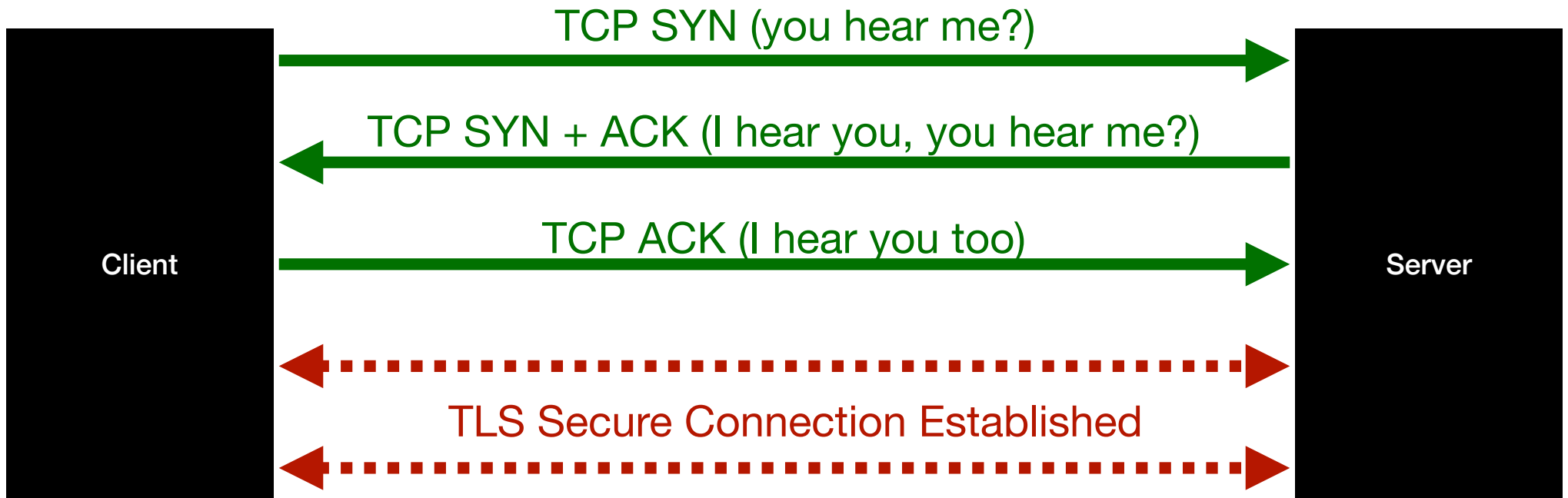
Domain - networking & reverse engineering

Activity - research & discovery

Fundamentals - HTTPS over TCP

Fundamentals in practice

HTTPS over TCP



Fundamentals in practice

Proxy detection - the issue

Our proxy server acts as a gateway to multiple proxy providers

Sometimes, we'd get detected when proxying through our server but not providers

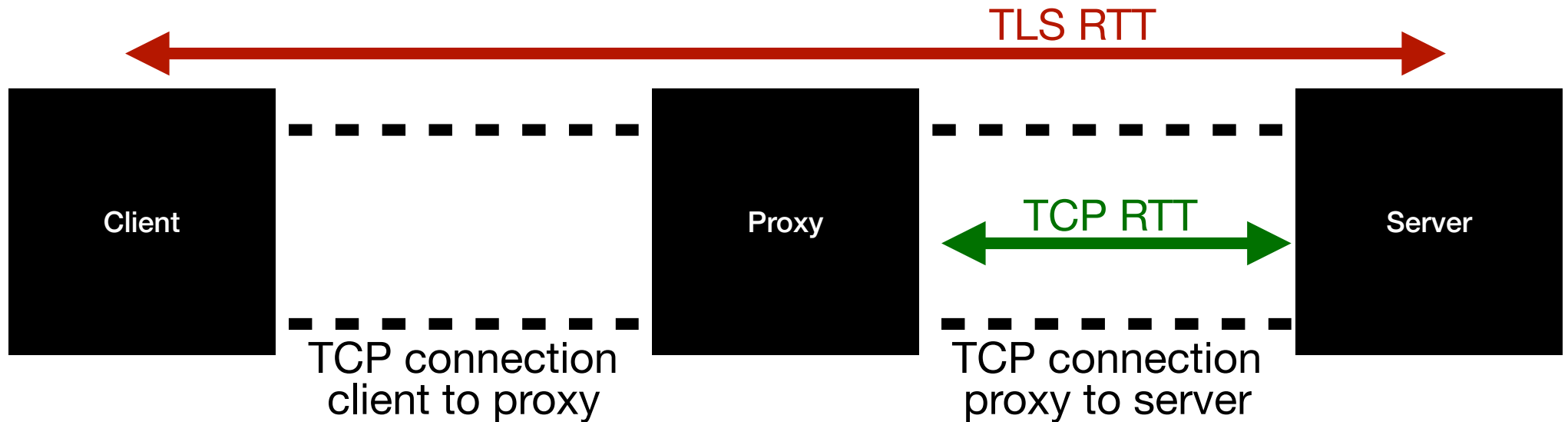
So, we wondered if our server added a *significant* time overhead

Currently, the server is a NodeJS app ("slow" but still good at IO)

Should we write a faster proxy server in Rust? This implies significant time & effort

Fundamentals in practice

Proxy detection - the discovery



From the server's POV, **TLS RTT** is way larger than **TCP RTT** (due to the proxy)

A faster proxy would probably still be detected

Fundamentals in practice

Case study 3 - overview

Layer of abstraction - AI coding agents

Activity - Adaptation & migration

Domain - Still working that out

Fundamentals - Still working that out

Fundamentals in practice

AI coding agents - the new layer of abstraction in town

The landscape has dramatically changed and is overwhelmingly fast-paced

We went from autocompletion to ChatGPT to OpenCode in only a few years

How do we adapt?

Which domains should we apply it to and how?

How do we migrate our current tech stack to better adapt to this new layer of abstraction?

Fundamentals in practice

AI coding agents - the new fundamentals in town

Co-founder
of OpenAI



Andrej Karpathy 
@karpathy

I've never felt this much behind as a programmer. The profession is being dramatically refactored as the bits contributed by the programmer are increasingly sparse and between. I have a sense that I could be 10X more powerful if I just properly string together what has become available over the last ~year and a failure to claim the boost feels decidedly like skill issue. There's a new programmable layer of abstraction to master (in addition to the usual layers below) involving agents, subagents, their prompts, contexts, memory, modes, permissions, tools, plugins, skills, hooks, MCP, LSP, slash commands, workflows, IDE integrations, and a need to build an all-encompassing mental model for strengths and pitfalls of fundamentally stochastic, fallible, unintelligible and changing entities suddenly intermingled with what used to be good old fashioned engineering. Clearly some powerful alien tool was handed around except it comes with no manual and everyone has to figure out how to hold it and operate it, while the resulting magnitude 9 earthquake is rocking the profession. Roll up your sleeves to not fall behind.

Fundamentals in practice

What next?

Fundamentals make you a better engineer *but* mastering them takes time & effort

Build something. Then, master the fundamentals involved

Share the knowledge and keep exploring

[frontend.mu](#), [cloudnativemauritius.com](#), [mscc.mu](#), [pymug.com](#), ...

That's it folks!

github.com/houzyk/talks

houzairmk@icloud.com

linkedin.com/in/houzairk/

thephilosophicalcode.com